

Architecture des systèmes numériques et informatiques

Organisation du cours :

Les enseignements se déroulent en plusieurs phases.

- 3 cours de 2h
- 3 séances de TD de 2h
- 5 séances de TP de 2h
- 1 séance d'examen de 2h

Objectifs (notions abordées) :

- Codage des nombres, des caractères, des images.
- Fonctions logiques – logique combinatoire et séquentielle
- Structure d'un processeur – Types de mémoires
- Périphériques d'entrées – sorties

PARTIE I :

Codage de l'information

Objectifs (objectifs pédagogiques) :

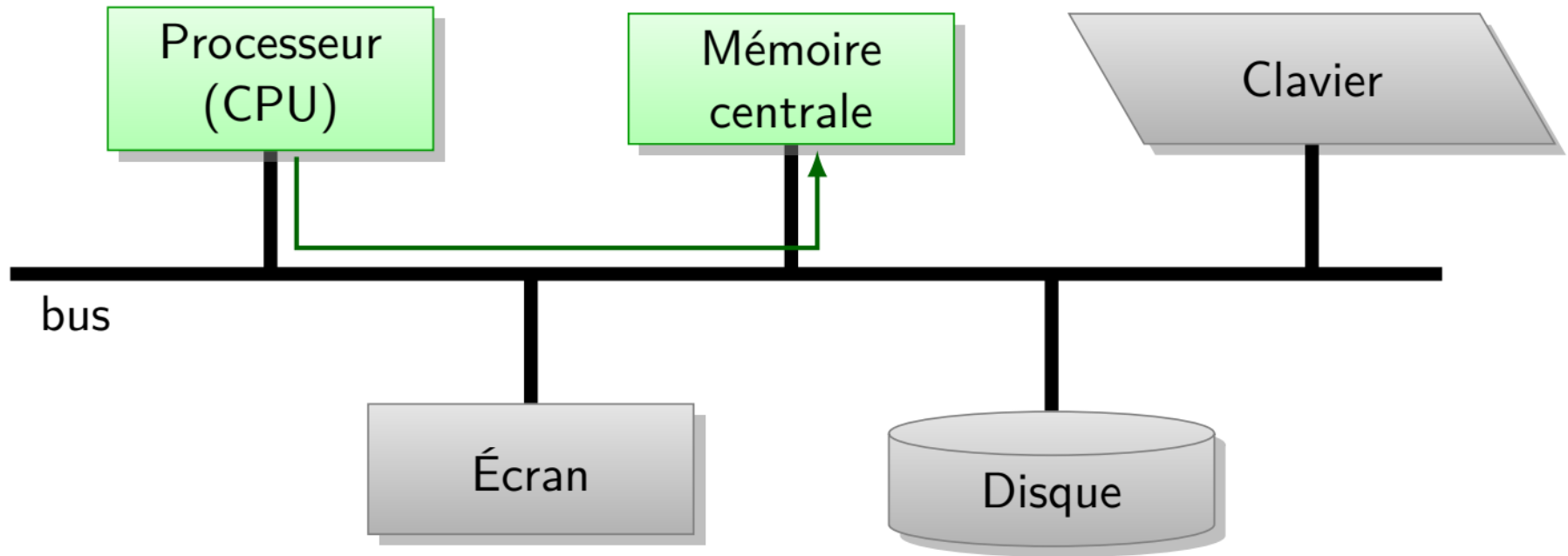
Codage des nombres

- Maîtriser la représentation des nombres dans les binaires, octales et hexadécimales,
- Maîtriser l'arithmétique binaire élémentaire : addition, soustraction et multiplication,

Codages des caractères et des images

- Maîtriser le codage des caractères : ASCII, UTF-8...
- Maîtriser la représentation des images numériques : bitmap, niveaux de gris, RGB,
- Maîtriser les notions de compression de données,

Ordinateur (représentation simplifiée) :



Gérer des Entrées/Sorties

Codage de l'information :

Un ordinateur manipule des données (textes, images, vidéos,)

Besoin de coder et représenter ces données,

De nature différente

- Des nombres,
- Des chaînes de caractères,
- Des images et des vidéos,
- Des audios,

De taille différente

- Taille fixe : numéro de téléphone, code postal
- Taille variable : nom, prénom, adresses, vidéos, texte

Codage de l'information (représentation binaire) :

- En électronique numérique : deux niveaux logiques,
- Mot binaire : groupe ordonné d'éléments binaires,
- Les éléments binaires sont nommés bits (binary digits),
- Valence M d'un code binaire de n bits : $M = 2^n$,
- Tailles normalisées de mots binaires :

n	M
8	256
16	65 536
24	16 777 216
32	4 294 967 296
64	$18 * 10^{18}$

Codage de l'information (codage des nombres):

Un but de calcul : codage des nombres,

Apparition du calcul

- Dès la préhistoire on comptait avec les doigts et les cailloux
- Calcul vient du latin calculus signifiant petits cailloux

Chaque civilisation (Grecs, Romains,...) avait développé des

- Systèmes et bases de numérotation pour représenter les nombres
- Méthodes pour compter et calculer

Codage de l'information (codage des nombres):

Un but de calcul : codage des nombres,

Origine des systèmes de numérotation

- Base 10 : nombre des doigts des 2 mains
- Base 20 : mains et pieds
- Base 12 : 3 phalanges et 4 doigts

Codage des nombres :

Un but de calcul : codage des nombres,

Bases de codage possible

- Base 10 : base de calcul usuelle
- Base 24 : heures
- Base 60 : minutes, secondes,....

Bases fréquentes

- Pour les êtres humains : base décimale
- Pour un ordinateur : base binaire et dérivées (octale et hexadécimale)

Origine : absence ou présence de courant électrique (0 ou 1),

Codage des nombres (le système décimal) :

Base de calcul usuelle pour les êtres humains,

Valeur en décimal (base 10) d'un entier naturel " $a_n a_{n-1} \dots a_1 a_0$ " codé dans la base 10 est :

$$a_n * 10^n + a_{n-1} * 10^{n-1} + \dots + a_1 * 10^1 + a_0 * 10^0$$

a_x : la valeur décimale de chaque chiffre

Exemple :

$$(2453)_{10} = 2 * 10^3 + 4 * 10^2 + 5 * 10^1 + 3 * 10^0$$

$$(253)_{10} = 2 * 10^2 + 5 * 10^1 + 3 * 10^0$$

Codage des nombres (le système binaire) :

Base de calcul pour un ordinateur,

Valeur en décimal (base 10) d'un entier naturel " $a_n a_{n-1} \dots a_1 a_0$ " codé dans la base 2 est :

$$a_n * 2^n + a_{n-1} * 2^{n-1} + \dots + a_1 * 2^1 + a_0 * 2^0$$

a_x : la valeur binaire de chaque chiffre

Exemple :

$$(1111)_2 = 1 * 2^3 + 1 * 2^2 + 1 * 2^1 + 1 * 2^0$$

$$(101)_2 = 1 * 2^2 + 0 * 2^1 + 1 * 2^0$$

Conversion binaire - décimal :

Binaire vers décimal

Pour calculer la valeur décimale N de « $a_n a_{n-1} \dots a_1 a_0$ » codé en base 2

$$N = a_n * 2^n + a_{n-1} * 2^{n-1} + \dots + a_1 * 2^1 + a_0 * 2^0$$

a_x : la valeur binaire de chaque chiffre

Exemple :

$$(1111)_2 = 1 * 2^3 + 1 * 2^2 + 1 * 2^1 + 1 * 2^0 = 15$$

$$(101)_2 = 1 * 2^2 + 0 * 2^1 + 1 * 2^0 = 5$$

Conversion binaire - décimal :

Décimal vers binaire

Pour calculer le code binaire « $a_n a_{n-1} \dots a_1 a_0$ » d'un décimal de valeur N :

- On procède par une série de divisions entières par 2
- Division du nombre du nombre décimal N par 2 : donne une valeur v_0 et un reste r_0
- On divise v_0 par 2 et on obtient v_1 et r_1
- On recommence la même opération avec les valeurs $v_1, \dots, v_{n-1}$ et on obtient les valeurs $v_2, \dots, v_n$ et les restes $r_2, \dots, r_n$ jusqu'à $v_n = 0$

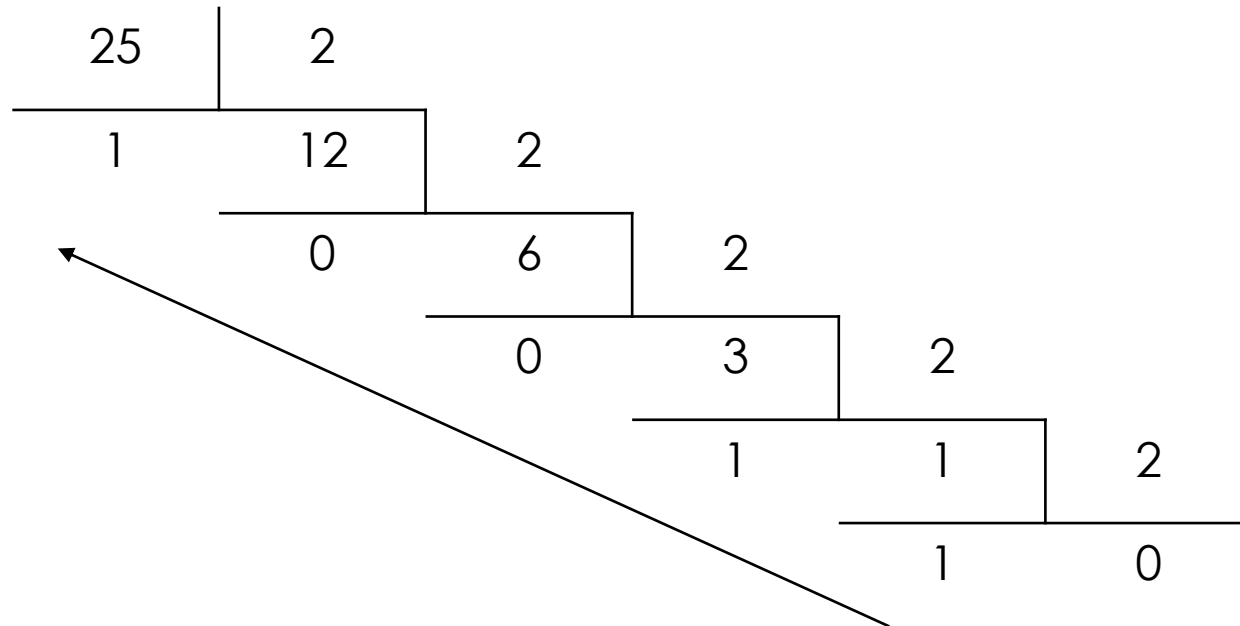
On obtient le code binaire de N qui est « $r_n r_{n-1} \dots r_1 r_0$ »

Conversion binaire - décimal :

Décimal vers binaire

Exemple :

$N = 25$



$$(N)_2 = 11001$$

Représentation des nombres dans une base b:

Pour une base b, il y a b symboles différents (les chiffres de la base)

Base 10 : 0, 1, 2, 3, 4, 5, 6, 7, 8, 9

Base 2 : 0, 1

Base 4 : 0, 1, 2, 3

Base 16 : 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, a, b, c, d, e, f

Un entier naturel N s'écrit dans la base b sous la forme :

$$N_{(b)} = a_n a_{n-1} \dots a_1 a_0 \quad \text{avec } a_x \text{ un des chiffres de la base b}$$

Exemple :

$$N_{(10)} = 15$$

$$N_{(2)} = 1111$$

$$N_{(16)} = F$$

Représentation des nombres dans une base b :

Tout nombre N exprimé dans une base b peut se décomposer sous la forme polynômiale :

$$N_b = a_n * b^n + a_{n-1} * b^{n-1} + \dots + a_1 * b^1 + a_0 * b^0 + a_{-1} * b^{-1} + \dots + a_{-m} * b^{-m}$$

Exemple :

$$133_{(10)} = \dots$$

$$13,12_{(10)} = \dots$$

$$101_{(2)} = \dots$$

$$11,01_{(2)} = \dots$$

Représentation des nombres dans une base b :

Tout nombre N exprimé dans une base b peut se décomposer sous la forme polynômiale :

$$N_b = a_n * b^n + a_{n-1} * b^{n-1} + \dots + a_1 * b^1 + a_0 * b^0 + a_{-1} * b^{-1} + \dots + a_{-m} * b^{-m}$$

Exemple :

$$133_{(10)} = 1 * 10^2 + 3 * 10^1 + 3 * 10^0$$

$$13,12_{(10)} = 1 * 10^1 + 3 * 10^0 + 1 * 10^{-1} + 2 * 10^{-2}$$

$$101_{(2)} = 1 * 2^2 + 0 * 2^1 + 1 * 2^0 = 5$$

$$11,01_{(2)} = 1 * 2^1 + 1 * 2^0 + 0 * 2^{-1} + 1 * 2^{-2} = 3,25$$

Conversion base b - décimal :

Base b vers décimal

Pour calculer la valeur décimale N de « $a_n a_{n-1} \dots a_1 a_0$ » codé en base b

$$N = a_n * b^n + a_{n-1} * b^{n-1} + \dots + a_1 * b^1 + a_0 * b^0$$

a_x : la valeur de chaque chiffre de N dans la base b

Exemple :

$$(123)_4 = 1 * 4^2 + 2 * 4^1 + 3 * 4^0 = 27$$

$$(1b)_{16} = 1 * 16^1 + 11 * 16^0 = 27$$

Conversion base b - décimal :

Décimal vers base b

Pour calculer le code « $a_n a_{n-1} \dots a_1 a_0$ » dans la base b d'un décimal de valeur N :

- On procède par une série de divisions entières par b
- Division du nombre du nombre décimal N par b : donne une valeur v_0 et un reste r_0
- On divise v_0 par b et on obtient v_1 et r_1
- On recommence la même opération avec les valeurs $v_1, \dots, v_{n-1}$ et on obtient les valeurs $v_2, \dots, v_n$ et les restes $r_2, \dots, r_n$ jusqu'à $v_n = 0$

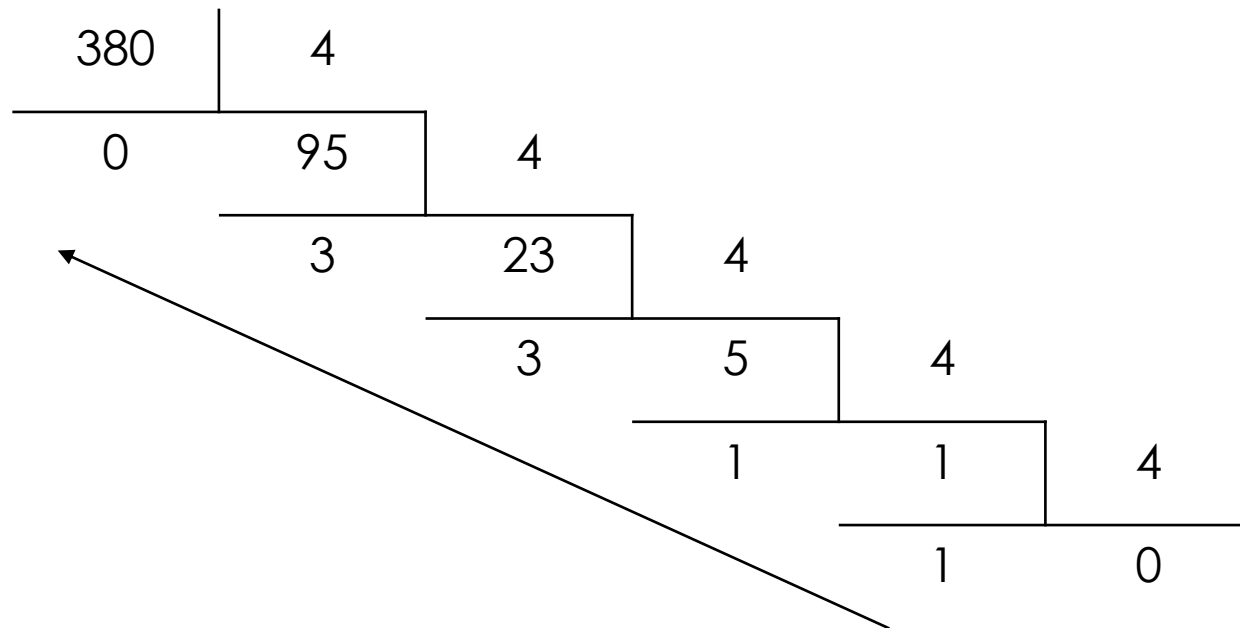
On obtient le code binaire de N qui est « $r_n r_{n-1} \dots r_1 r_0$ »

Conversion base b - décimal :

Décimal vers base b

Exemple :

$N = 380$



$$(N)_4 = 11330$$

Conversion base b - décimal :

Décimal vers base b

Exemple :

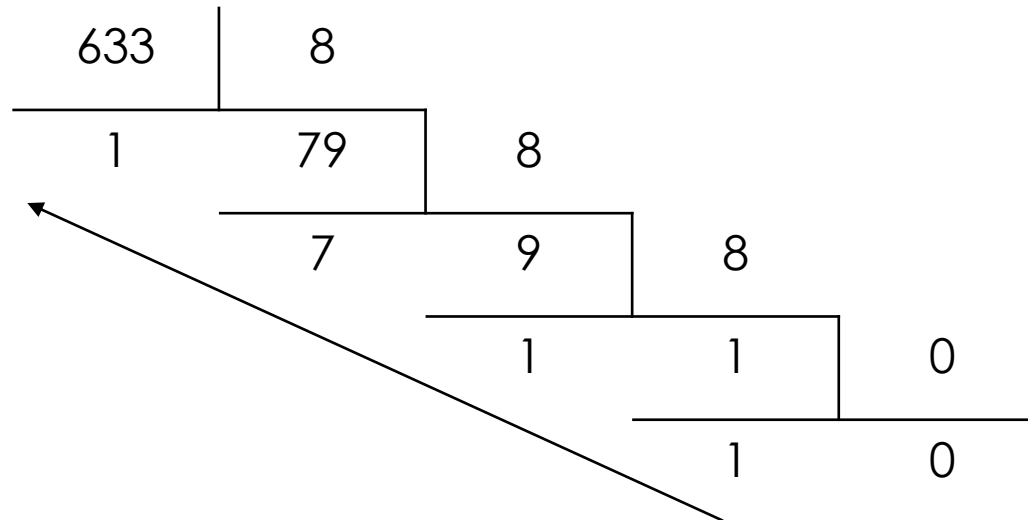
$N = 633$ base 8 ?

Conversion base b - décimal :

Décimal vers base b

Exemple :

$N = 633$



$$(N)_8 = 1171$$

Conversion d'une base à une autre :

Pour convertir d'une base b_1 à une autre base b_2 :

- b_1 vers décimal
- Décimal vers b_2

Exemple :

$$(N)_4 = 11330$$

$$(N)_8 = ???$$

Conversion d'une base à une autre :

Pour convertir d'une base b_1 à une autre base b_2 :

- b_1 vers décimal
- Décimal vers b_2

Exemple :

$$(N)_4 = 11330$$

$$(N)_8 = 574$$

Systèmes octal et hexadécimal :

Conversion du binaire à l'octal ou hexadécimal:

- 1 chiffre octal = un groupe de 3 chiffres binaires
- 1 chiffre hexadécimal = un groupe de 4 chiffres binaires
- $(000)_2 = (0)_8, (001)_2 = (1)_8, \dots, (111)_2 = (7)_8$ avec 3 bits on peut coder les 8 chiffres de la base octale
- $(0000)_2 = (0)_{16}, (0001)_2 = (1)_{16}, \dots, (1111)_2 = (F)_{16}$ avec 4 bits on peut coder les 16 chiffres de la base hexadécimale

Systèmes octal et hexadécimal :

Conversion du binaire à l'octal ou hexadécimal:

- 1 chiffre octal = un groupe de 3 chiffres binaires
- 1 chiffre hexadécimal = un groupe de 4 chiffres binaires

Exemple :

$$N_{(2)} = (10110001101011)_2$$

$$N_{(4)} = \dots$$

$$N_{(8)} = \dots$$

$$N_{(16)} = \dots$$

Systèmes octal et hexadécimal :

Conversion du binaire à l'octal ou hexadécimal:

- 1 chiffre octal = un groupe de 3 chiffres binaires
- 1 chiffre hexadécimal = un groupe de 4 chiffres binaires

Exemple :

$$N_{(4)} = (2301)_4$$

$$N_{(2)} = \dots$$

$$N_{(8)} = (5273)_8$$

$$N_{(2)} = \dots$$

$$N_{(16)} = (2C01)_{16}$$

$$N_{(2)} = \dots$$

Addition binaire :

Même principe qu'en décimal

$$\begin{array}{rcll} & 52 & \longrightarrow & \overset{1}{0} \overset{1}{0} \overset{1}{1} \overset{1}{1} 0 1 0 0 \\ + & 45 & \longrightarrow & 0 0 1 0 1 1 0 1 \\ \hline = & 97 & \longrightarrow & 0 1 1 0 0 0 0 1 \end{array}$$

$$\begin{array}{rcll} & 13 & \longrightarrow & \overset{1}{1} 1 0 1 \\ + & 10 & \longrightarrow & 1 0 1 0 \\ \hline = & & \longrightarrow & 1 0 1 1 1 \end{array}$$

Attention au dépassement de capacité (problème de débordement)

Addition binaire :

Même principe qu'en décimal

$$\begin{array}{rcl} & 13 & \longrightarrow & & & & 1 & 1 & 0 & 1 \\ + & 10 & \longrightarrow & & & & 1 & 0 & 1 & 0 \\ \hline = & & \longrightarrow & 1 & 0 & 1 & 1 & 1 & & \end{array}$$

Débordement : la taille allouée au codage est trop petite pour coder le résultat d'un calcul

Exemple avec une addition sur 8 bits: $10101010 + 10000000 = 100101010$

Besoin de 9 bits pour coder le résultat

Soustraction binaire :

La soustraction binaire peut se faire comme en décimal

$$\begin{array}{rcll} & 52 & \longrightarrow & 0\ 0\ 1\ 1\ 0\ 1\ 0\ 0 \\ - & 45 & \longrightarrow & 0\ 0\ 1\ 0\ 1\ 1\ 0\ 1 \\ \hline = & 7 & \longrightarrow & 0\ 0\ 0\ 0\ 0\ 1\ 1\ 1 \end{array}$$

$$\begin{array}{rcll} & 13 & \longrightarrow & 1\ 1\ 0\ 1 \\ - & 10 & \longrightarrow & 1\ 0\ 1\ 0 \\ \hline = & 3 & \longrightarrow & 0\ 0\ 1\ 1 \end{array}$$

Autre technique : utiliser les compléments à 2 et ne faire que des additions

Les nombres signés:

Deux représentations possibles :

- Signe + magnitude
 - 1 bit de signe (poids fort) : 0 entier positif, 1 entier négatif
 - $2^{w-1} - 1$ valeurs positives et $2^{w-1} - 1$ valeurs négatives
 - 2 représentations pour le 0
 - Additionneurs différents suivant le signe
- complément à deux
 - $2^{w-1} - 1$ valeurs positives et 2^{w-1} valeurs négatives
 - une seule représentation pour le 0
 - Additionneur unique

Les nombres signés:

Complément à 2 (complément arithmétique) :

- Complément logique d'un nombre binaire
 - Les 1 deviennent des 0 et les 0 deviennent des 1
 - Le complément logique est dit « complément à 1 »
- Complément arithmétique d'un nombre binaire
 - Complément logique du nombre auquel on rajoute la valeur de 1
 - Le complément arithmétique est dit « complément à 2 »
- Codage des nombres signés avec complément à 2
 - Nombre positif : comme pour un entier non signé
 - Nombre négatif : complément à 2 de son opposé positif
 - Bit de poids fort code le signe : 0 pour positif et 1 pour négatif

Exemple : $6 = 0110$

$-6 = 1010$

Addition/soustraction binaire en signé :

Codage en complément à 2 :

- Simplifie les additions et les soustractions
- On peut additionner directement les nombres, quels que soit leurs signes, le résultat sera directement correct (si pas de débordement)
- Soustraction d'un nombre équivalente à l'addition de son complément à 2
 - $A - B = A + \text{comp2}(B)$
 - Valable dans tous les cas, quels que soient les signes de A et B

Exemple :

		0	0					
	5		0	0	1	0	1	
+	-8		1	1	0	0	0	
	-3		0	1	1	1	0	1

Addition/soustraction binaire en signé :

Codage en complément à 2 :

- Simplifie les additions et les soustractions
- On peut additionner directement les nombres, quels que soit leurs signes, le résultat sera directement correct (si pas de débordement)
- Soustraction d'un nombre équivalente à l'addition de son complément à 2
 - $A - B = A + \text{comp2}(B)$
 - Valable dans tous les cas, quels que soient les signes de A et B

Exemple :

			1	1		
	8		0	1	0	0
+	-5		1	1	0	1
	3		1	0	0	1

Addition/soustraction binaire en signé :

Codage en complément à 2 :

- Gestion des débordements différent de l'addition non signée
 - On regarde les retenues des deux derniers bits additionner pour savoir s'il y a eu un débordement
 - Si les retenues sont identiques : pas de débordement
 - Si les retenues sont différentes : débordement
 - On néglige systématiquement la retenue sur le bit supplémentaire pour déterminer le résultat final

Exemple :

$$\begin{array}{rcccccc} & & & & & & 1 & 1 \\ & 8 & & 0 & 1 & 0 & 0 & 0 \\ + & -5 & & 1 & 1 & 0 & 1 & 1 \\ \hline & 3 & & 1 & 0 & 0 & 0 & 1 & 1 \end{array}$$

Addition/soustraction binaire en signé :

Codage en complément à 2 :

- Gestion des débordements différent de l'addition non signée
 - On regarde les retenues des deux derniers bits additionner pour savoir s'il y a eu un débordement
 - Si les retenues sont identiques : pas de débordement
 - Si les retenues sont différentes : débordement
 - On néglige systématiquement la retenue sur le bit supplémentaire pour déterminer le résultat final

Exemple :

$$\begin{array}{rcccccc} & & 0 & 1 & & & \\ & 8 & & & & & \\ + & 9 & & 0 & 1 & 0 & 0 & 0 & 1 \\ \hline -15 & & 0 & 1 & 0 & 0 & 0 & 0 & 1 \end{array}$$

Codage des nombres réels à valeur dans R:

2 formats sont possibles :

- Virgule fixe
- Virgule flottante

Virgule fixe :

Pour pouvoir effectuer des opérations arithmétiques, il faut savoir où est placée la virgule Q_m

m bits pour la partie fractionnaire

Résolution : $q = b^{-m}$

Dynamique limitée par la précision et le nombre de chiffres total

$$N_{(b)} = a_n * b^n + a_{n-1} * b^{n-1} + \dots + a_1 * b^1 + a_0 * b^0 + a_{-1} * b^{-1} + \dots + a_{-m} * b^{-m}$$

Pour les nombres non-signés au format (b, n, m)

$$\Delta = b^{n+1} - b^{-m} \text{ et } \delta = b^{-m}$$

Virgule flottante :

Principe et intérêt :

- Avoir une virgule flottante et une précision limitée
- Ne coder que les chiffres significatifs
- $N = +/- M * b^E$
 - N : nombre codé
 - M (mantisse) : nombre de X chiffres de la base b
 - E (exposant) : nombre de Y chiffres de la base b
 - +/- : codage du signe
- Le nombre est présenté sous la forme normalisée pour déterminer la mantisse et l'exposant
 - Pas de chiffre avant la virgule : $0,XXXXXX * b^E$

PARTIE II :

Fonctions logiques et algèbre de Boole

Fonctions logiques de bases :

Valeur booléenne

- 2 valeurs possibles {vrai, faux}
- Opérations logiques
- Fonction logique $s = s(a_1, a_2, \dots, a_n)$

Table de vérité

- Donne l'état logique de la fonction pour chacune des combinaisons des états des variables
- Une fonction logique à N entrées sera représentée par une table à 2^N lignes

Fonctions logiques de bases :

Table de vérité

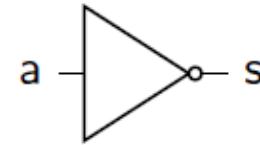
- Donne l'état logique de la fonction pour chacune des combinaisons des états des variables
- Une fonction logique à N entrées sera représentée par une table à 2^N lignes
- $s = s(a_1, a_2)$

a_1	a_2	S
0	0	X
0	1	X
1	0	X
1	1	X

Fonctions logiques de bases :

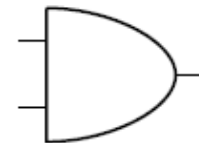
- NON (NOT) : $s(a) = \bar{a}$

a	s
0	1
1	0



- ET (AND) : $s(a_1, a_2) = a_1 \cdot a_2$

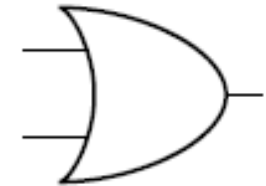
a_1	a_2	s
0	0	0
0	1	0
1	0	0
1	1	1



Fonctions logiques de bases :

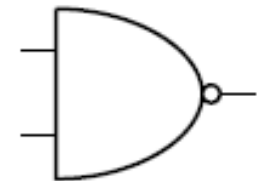
- OU (OR) : $s(a_1, a_2) = a_1 + a_2$

a_1	a_2	s
0	0	0
0	1	1
1	0	1
1	1	1



- NON ET (NAND) : $s(a_1, a_2) = \overline{a_1 \cdot a_2}$

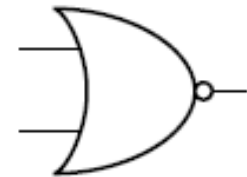
a_1	a_2	s
0	0	1
0	1	1
1	0	1
1	1	0



Fonctions logiques de bases :

- NON OU (NOR) : $s(a_1, a_2) = \overline{a_1 + a_2}$

a_1	a_2	s
0	0	1
0	1	0
1	0	0
1	1	0



- OU EXCLUSIF (XOR) : $s(a_1, a_2) = a_1 \oplus a_2 = a_1 \cdot \overline{a_2} + \overline{a_1} \cdot a_2$

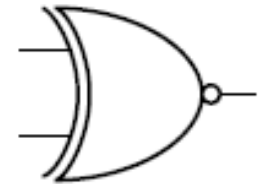
a_1	a_2	s
0	0	0
0	1	1
1	0	1
1	1	0



Fonctions logiques de bases :

- XNOR : $s(a_1, a_2) = \overline{a_1 \oplus a_2} = a_1 \cdot a_2 + \overline{a_1} \cdot \overline{a_2}$

a_1	a_2	s
0	0	1
0	1	0
1	0	0
1	1	1



Exemple :

Représentez sous schéma de symboles logiques interconnectées la fonction

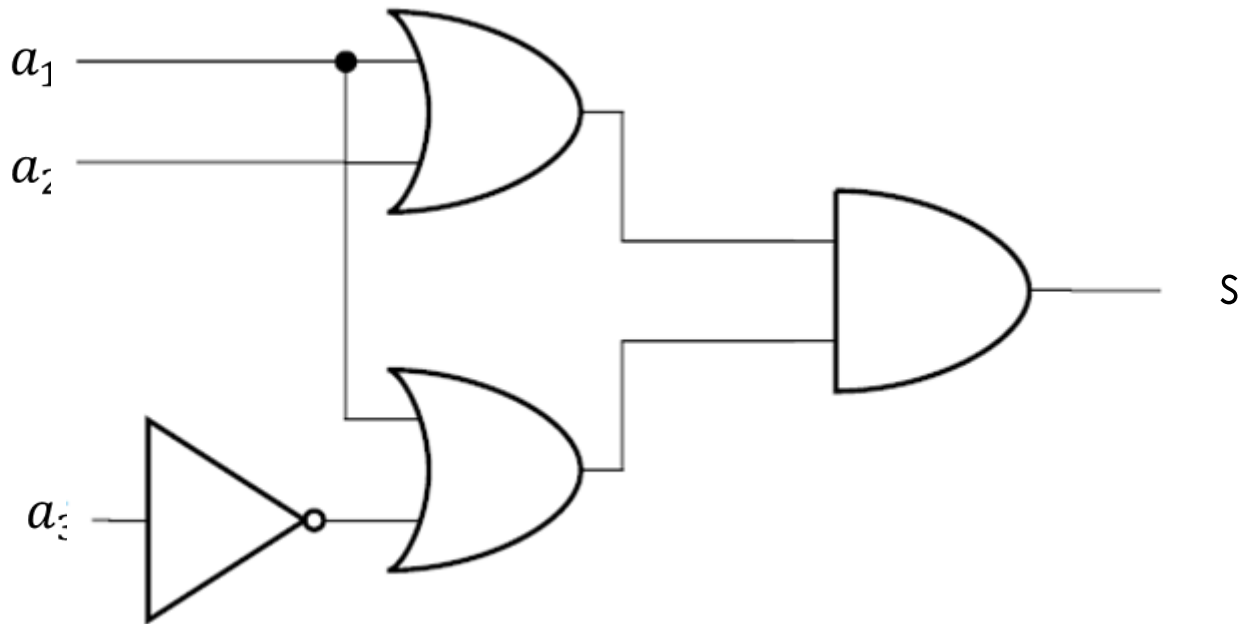
- $s(a_1, a_2, a_3) = (a_1 + a_2) \cdot (a_1 + \overline{a_3})$

Fonctions logiques de bases :

Exemple :

Représentez sous schéma de symboles logiques interconnectées la fonction

- $s(a_1, a_2, a_3) = (a_1 + a_2) \cdot (a_1 + \overline{a_3})$



Algèbre de Boole :

Initié par George Boole en 1854

Variables binaires ou booléennes : 0 ou 1 (proposition vrai $\Rightarrow$ 1 et proposition fausse $\Rightarrow$ 0)

Approche algébrique de la logique

- Définit les outils nécessaires à la description et l'établissement des relations entrées/sorties d'un système logique,

Propriétés des opérateurs de l'algèbre de Boole :

- Involution

$$\overline{\overline{a}} = a$$

- Éléments neutres

$$a + 0 = a \qquad a \cdot 1 = a$$

- Éléments absorbants

$$a + 1 = 1 \qquad a \cdot 0 = 0$$

- Complémentarité

$$a + \bar{a} = 1 \qquad a \cdot \bar{a} = 0$$

Propriétés des opérateurs de l'algèbre de Boole :

- Associativité

$$a + b + c = (a + b) + c = a + (b + c)$$

$$a . b . c = (a . b) . c = a . (b . c)$$

- Commutativité

$$a + b = b + a$$

$$a . b = b . a$$

- Distributivité

$$a . (b + c) = a . b + a . c$$

$$a + (b . c) = (a + b) . (a + c)$$

Propriétés des opérateurs de l'algèbre de Boole :

- Idempotence

$$a + a + \dots + a = a$$

$$a . a \dots a = a$$

- Simplification

$$a + (\bar{a} . b) = a + b$$

$$a . (\bar{a} + b) = a . b$$

- Redondance

$$a . b + \bar{a} . c + b . c = a . b + \bar{a} . c$$

$$(a + b) . (\bar{a} + c) . (b + c) = (a + b) . (\bar{a} + c)$$

Algèbre de Boole :

Complément d'une somme

- $\overline{a + b} = \bar{a} . \bar{b}$

a	b	a+b	$\overline{a + b}$	$\bar{a}$	$\bar{b}$	$\bar{a} . \bar{b}$
0	0	0	1	1	1	1
0	1	1	0	1	0	0
1	0	1	0	0	1	0
1	1	1	0	0	0	0

Complément de l'intersection

- $\overline{a . b} = \bar{a} + \bar{b}$

a	b	a . b	$\overline{a . b}$	$\bar{a}$	$\bar{b}$	$\bar{a} + \bar{b}$
0	0	0	1	1	1	1
0	1	0	1	1	0	1
1	0	0	1	0	1	1
1	1	1	0	0	0	0

Fonctions logiques :

- Prend en entrée une ou plusieurs variables booléennes
- Retourne en sortie une valeur booléenne en fonction des variables d'entrées
- Deux façons pour définir une fonction logique
 - Par une expression logique
 - Combinaison des variables d'entrées de la fonction via les opérateurs de base de l'algèbre de Boole
 - Plusieurs expressions possibles pour la même fonction
 - Par sa table de vérité
 - Table unique définissant la valeur de la fonction pour chaque combinaison possible des variables d'entrées

Fonctions logiques :

Exemple :

$$f_1(a, b) = a + b$$

$$f_2(a, b) = a + b + b$$

a	b	$f_1(a, b)$	$f_2(a, b)$
0	0	0	0
0	1	1	1
1	0	1	1
1	1	1	1

Fonctions logiques :

- Passage entre l'expression logique et la table de vérité de la fonction
 - On peut toujours déterminer l'une à partir de l'autre
- Deux fonctions logiques sont identiques si
 - Leurs tables de vérité sont identiques
 - On peut montrer via les propriétés de l'algèbre de Boole que leurs expressions logiques sont identiques

Fonctions logiques :

Exemple :

$$f_1(a, b) = a + b$$

$$f_2(a, b) = a + b + b$$

a	b	$f_1(a, b)$	$f_2(a, b)$
0	0	0	0
0	1	1	1
1	0	1	1
1	1	1	1

Fonctions logiques :

- Pour une fonction logique à x variables
 - Un minterme est un groupe des x variables liés par des ET (and)
 - Un maxterme est un groupe des x variables liés par des OU (OR)
- Deux formes canoniques pour une fonction logique
 - Première forme : union (OR) des mintermes
 - Seconde forme : intersection (AND) des maxtermes
- Unique expression possible dans chaque type de forme canonique pour une fonction logique donnée

Fonctions logiques :

Exemple (fonction logique avec 3 variables d'entrées)

Mintermes :

$$a.b.c \quad a.\bar{b}.c \quad \bar{a}.b.\bar{c} \dots$$

Maxtermes :

$$a + b + c \quad a + \bar{b} + c \quad \bar{a} + b + \bar{c} \dots$$

Première forme canonique :

$$f_1(a, b, c) = a.b.c + a.\bar{b}.c + \bar{a}.b.\bar{c}$$

Seconde forme canonique :

$$f_2(a, b, c) = (a + b + c). (a + \bar{b} + c). (\bar{a} + b + \bar{c})$$

Fonctions logiques :

Passage aux formes canoniques

- Partir de la fonction et la transformer pour faire apparaître les mintermes ou les maxtermes complets
- Rajouter les variables manquantes
- Utiliser les propriétés de l'algèbre de Boole
 - $a + \bar{a} = 1$ pour passer à la première forme canonique
 - $\bar{\bar{a}} = a$ et $a + \bar{a} = 1$ pour passer à la seconde forme canonique

Fonctions logiques :

Exemple :

- Première et seconde formes canonique de la fonction logique $f_1(a, b, c)$

$$f_1(a, b, c) = a.b + b.\bar{c}$$

Fonctions logiques :

Exemple :

Première et seconde formes canonique de la fonction logique $f_1(a, b, c)$

$$f_1(a, b, c) = a.b + b.\bar{c}$$

Première forme canonique :

$$f_1(a, b, c) = a.b + b.\bar{c}$$

$$= a.b.(c + \bar{c}) + (a + \bar{a}).b.\bar{c}$$

$$= a.b.c + a.b.\bar{c} + a.b.\bar{c} + \bar{a}.b.\bar{c}$$

$$= a.b.c + a.b.\bar{c} + \bar{a}.b.\bar{c}$$

Fonctions logiques :

Exemple :

Première et seconde formes canonique de la fonction logique $f_1(a, b, c)$

$$f_1(a, b, c) = a.b + b.\bar{c}$$

Seconde forme canonique :

$$f_1(a, b, c) = a.b + b.\bar{c}$$

$$\overline{f_1(a, b, c)} =$$

$$f_1(a, b, c) = \overline{\overline{f_1(a, b, c)}} =$$

Fonctions logiques :

Passage de la fonction logique à la table de vérité

- Pour chaque combinaison de valeurs possibles des variables d'entrées, on détermine la valeur booléenne de la fonction

Exemple :

Déterminer la table de vérité de la fonction :

$$f_1(a, b, c) = a.b + b.\bar{c}$$

Fonctions logiques :

Exemple :

$$f_1(a, b, c) = a.b + b.\bar{c}$$

a	b	c	$\bar{a}$	$\bar{b}$	$\bar{c}$	$a.b$	$b.\bar{c}$	$f_1(a, b, c)$
0	0	0	1	1	1	0	0	0
0	0	1	1	1	0	0	0	0
0	1	0	1	0	1	0	1	1
0	1	1	1	0	0	0	0	0
1	0	0	0	1	1	0	0	0
1	0	1	0	1	0	0	0	0
1	1	0	0	0	1	1	1	1
1	1	1	0	0	0	1	0	1

Fonctions logiques :

Passage de la table de vérité à la première forme canonique de la fonction logique

- Pour chaque valeur booléenne de la fonction qui est égale à 1
 - On définit un minterme de toutes les variables tels que si une variable $a = 1$, on note a , sinon on note $\bar{a}$
 - La forme canonique de la fonction est le OU (OR) de ces mintermes

Exemple :

Déterminer la première forme canonique de la fonction f_1 à partir de sa table de vérité :

$$f_1(a, b, c) = a.b + b.\bar{c}$$

Fonctions logiques :

Exemple :

Déterminer la première de la fonction f_1 à partir de sa table de vérité :

$$f_1(a, b, c) = a.b + b.\bar{c}$$

a	b	c	$\bar{a}$	$\bar{b}$	$\bar{c}$	$a.b$	$b.\bar{c}$	$f_1(a, b, c)$
0	0	0	1	1	1	0	0	0
0	0	1	1	1	0	0	0	0
0	1	0	1	0	1	0	1	1
0	1	1	1	0	0	0	0	0
1	0	0	0	1	1	0	0	0
1	0	1	0	1	0	0	0	0
1	1	0	0	0	1	1	1	1
1	1	1	0	0	0	1	0	1

Simplification des fonctions logiques :

- Les formes canoniques d'une fonction logique sont une définition correcte de la fonction
- Ecrire la même fonction avec moins de termes et moins d'occurrences des variables
- Moins de termes et moins d'occurrences équivaut à un circuit moins complexe avec moins de portes logiques
- Deux méthodes de simplification :
 - Propriétés de l'algèbre de Boole
 - Les tableaux de Karnaugh

Simplification des fonctions logiques :

- Propriétés de l'algèbre de Boole
 - Simplifier la fonction logique en utilisant les propriétés de l'algèbre de Boole
 - Chercher la forme la plus simplifiée possible en simplifiant la fonction simplifiée

Exemple :

$$f_1(a, b, c) = a.b.c + a.b.\bar{c} + a.\bar{b}.c + a.\bar{b}.\bar{c}$$

Simplification des fonctions logiques :

- Propriétés de l'algèbre de Boole
 - Simplifier la fonction logique en utilisant les propriétés de l'algèbre de Boole
 - Chercher la forme la plus simplifiée possible en simplifiant la fonction simplifiée

Exemple :

$$f_1(a, b, c) = a.b.c + a.b.\bar{c} + a.\bar{b}.c + a.\bar{b}.\bar{c}$$

$$f_1(a, b, c) = a$$

Simplification des fonctions logiques :

- Tableaux de Karnaugh
 - Représentation sous forme particulière de la table de vérité de la fonction logique (on représente un tableau à 2 dimensions avec une seule variable qui change d'une colonne et une colonne adjacente et d'une ligne à une ligne adjacente)
 - On regroupe en blocs rectangulaires de 2^n les 1 adjacents (il faut créer les blocs les plus gros possible)
 - La fonction simplifiée correspond au OU (OR) des termes correspondants à chaque bloc de 1
 - Le terme de chaque bloc correspond au ET (AND) des variables inchangées dans un bloc

Simplification des fonctions logiques :

- Tableaux de Karnaugh

Exemple :

$$f_1(a, b, c) = a.b.c + a.b.\bar{c} + a.\bar{b}.c + a.\bar{b}.\bar{c}$$

$a \backslash bc$	00	01	11	10
0	0	0	0	0
1	1	1	1	1

$$f_1(a, b, c) = a$$

Simplification des fonctions logiques :

- Tableaux de Karnaugh

Exemple :

$$f_1(a, b, c) = a.b.c + a.b.\bar{c} + a.\bar{b}.c + a.\bar{b}.\bar{c} + \bar{a}.\bar{b}.c$$

Simplification des fonctions logiques :

- Tableaux de Karnaugh

Exemple :

$$f_1(a, b, c) = a.b.c + a.b.\bar{c} + a.\bar{b}.c + a.\bar{b}.\bar{c} + \bar{a}.\bar{b}.c$$

$a \backslash bc$	00	01	11	10
0	0	1	0	0
1	1	1	1	1

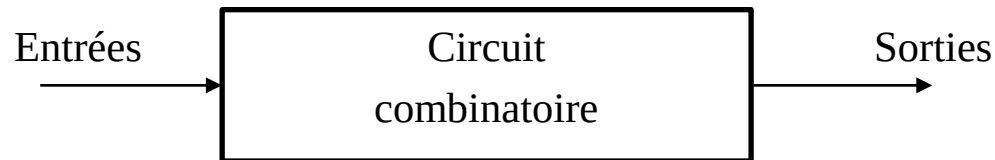
$$f_1(a, b, c) = a + \bar{b}c$$

Logique combinatoire et logique séquentiel :

- Un circuit numérique (logique) est un réseau de composants qui traitent des données à valeurs discrètes et produit des résultats également à valeurs discrètes,
- Un circuit numérique est défini par :
 - Ses entrées
 - Ses sorties
 - La spécification (sa fonction logique)
- Deux types :
 - Logique combinatoire
 - Logique séquentielle

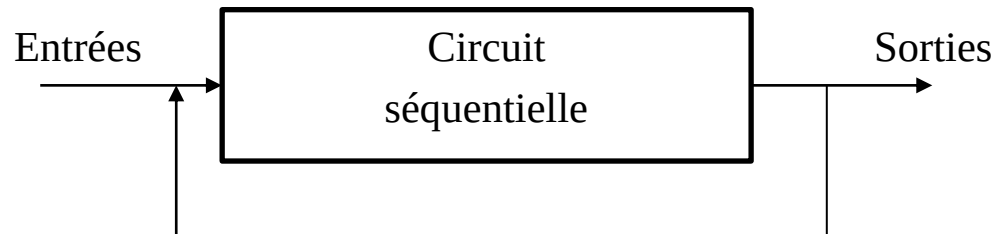
Logique combinatoire et logique séquentiel :

- Circuit combinatoire :
 - C'est l'absence de mémoire qui les définissent
 - Les sorties sont une fonction combinatoire des entrées
 - Une configuration des entrées correspond à une unique configuration des sorties



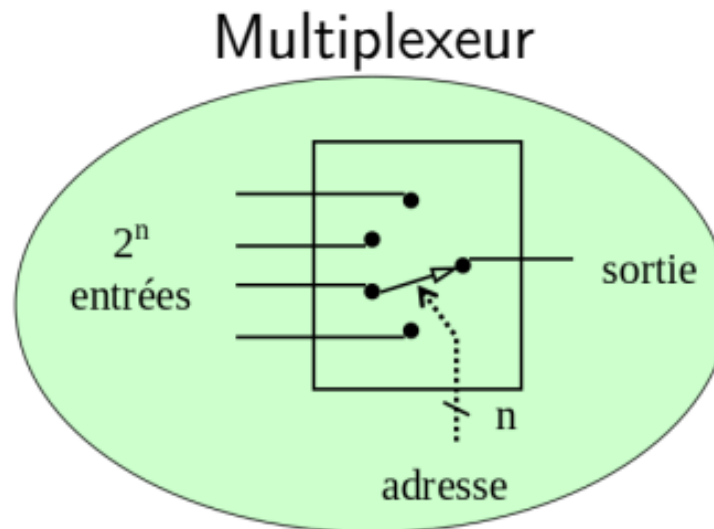
Logique combinatoire et logique séquentiel :

- Circuit séquentielle :
 - Les sorties sont en fonction des entrées mais aussi de l'état interne du système
 - Une configuration des entrées peut correspondre à plusieurs configurations des sorties



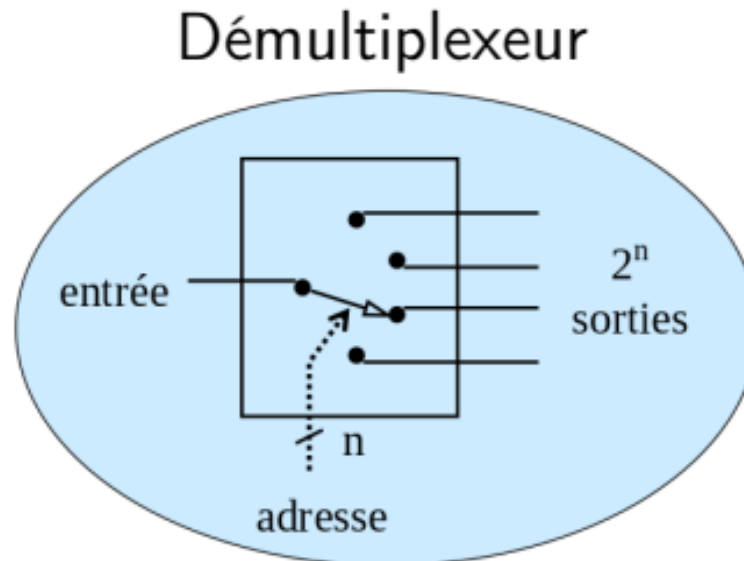
Logique combinatoire :

- Les Multiplexeurs :
 - Aiguille un signal d'entrées parmi 2^n vers une sortie grâce à n bits d'adresse



Logique combinatoire :

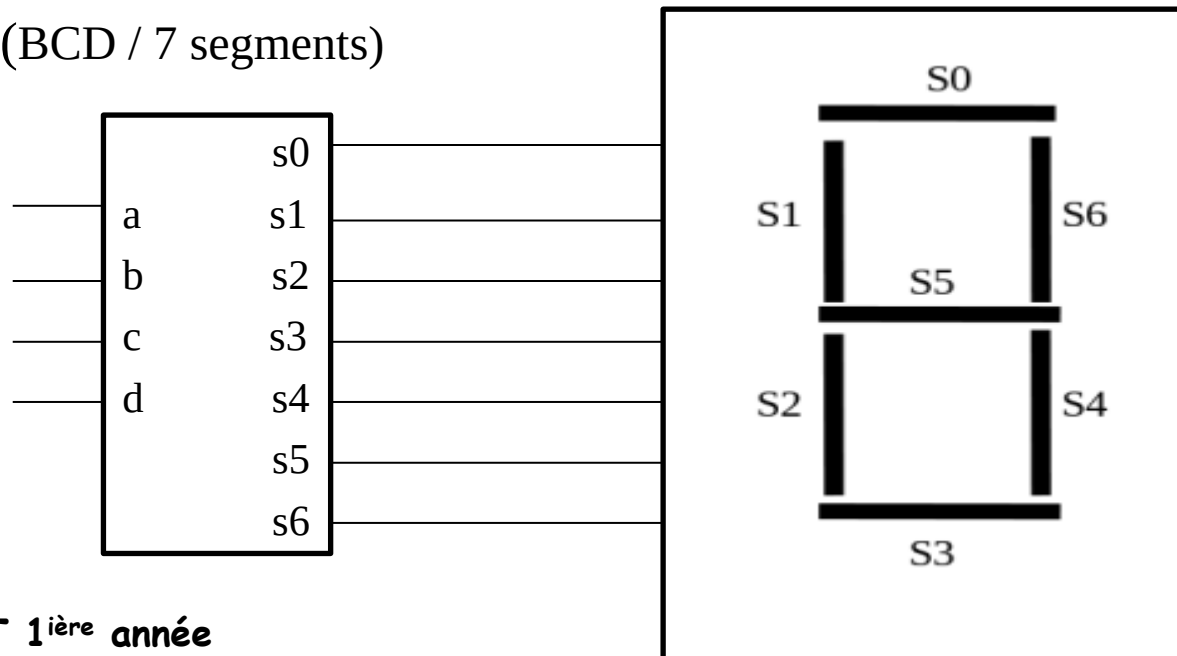
- Les Démultiplexeurs :
 - Aiguille un signal d'entrées vers une sortie parmi 2^n grâce à n bits d'adresse



Logique combinatoire :

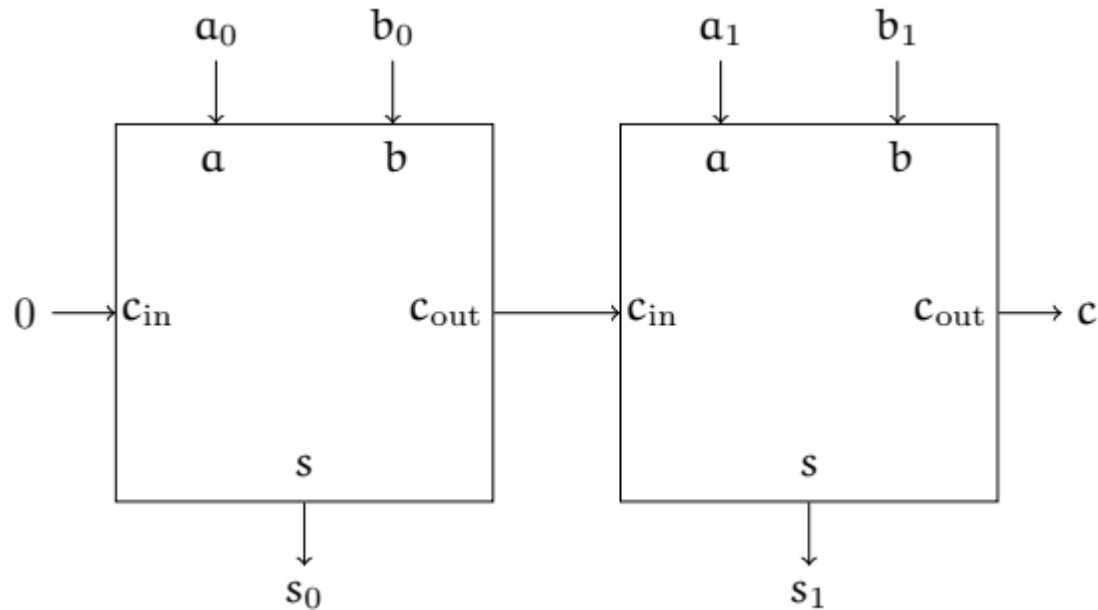
- Les transcodeurs :
 - Transforme une information présente en entrée sous une forme donnée (code 1) en la même information en sortie mais sous une autre forme (code 2)

Exemple : (BCD / 7 segments)



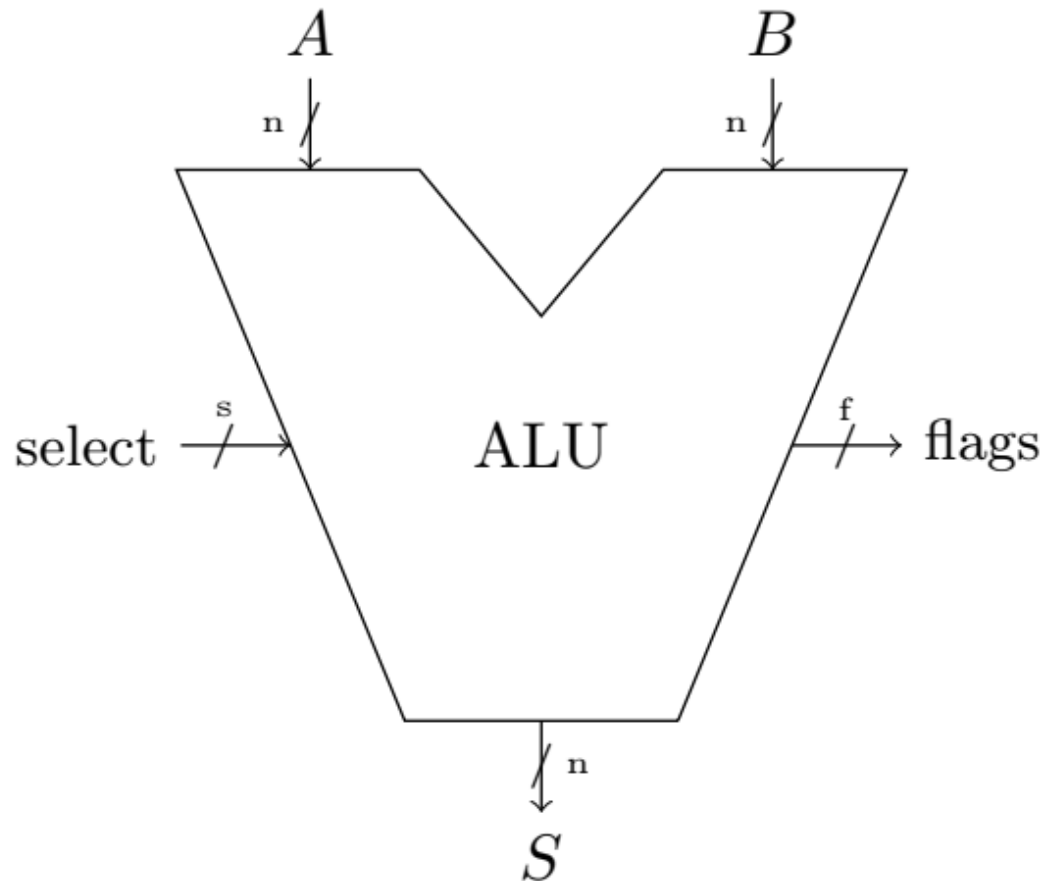
Logique combinatoire :

- Additionneurs :
 - Prend en entrée 2 bits et une retenue et délivre leur somme et la retenue de la somme



Logique combinatoire :

- L'unité arithmétique et logique (ALU) :

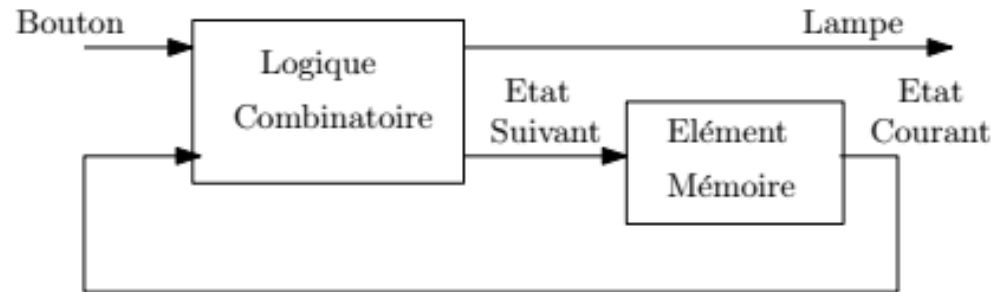


Logique combinatoire :

- L'unité arithmétique et logique (ALU) :
 - 2 entrées A et B codées sur 4 bits
 - Une sortie S codée sur 4 bits
 - Une sortie f qui vaut 1 si un débordement est détecté sur S
 - Une entrée *select* codée sur 2 bits permettant de sélectionner l'opération à effectuer :
 - Addition (00)
 - Soustraction (01)
 - Ou (10)
 - Et (11)

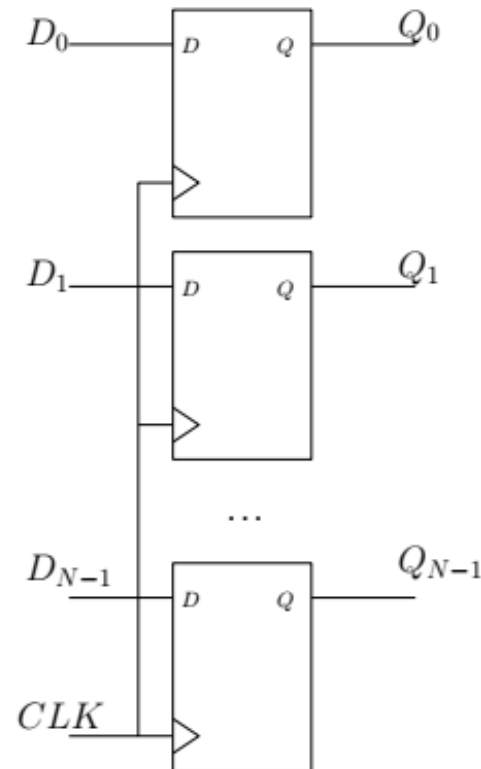
Logique séquentielle :

- Système séquentiel :
 - La logique séquentielle permet de stocker l'état courant
 - La logique combinatoire permet de calculer l'état futur et les sorties à partir de l'état présent des entrées

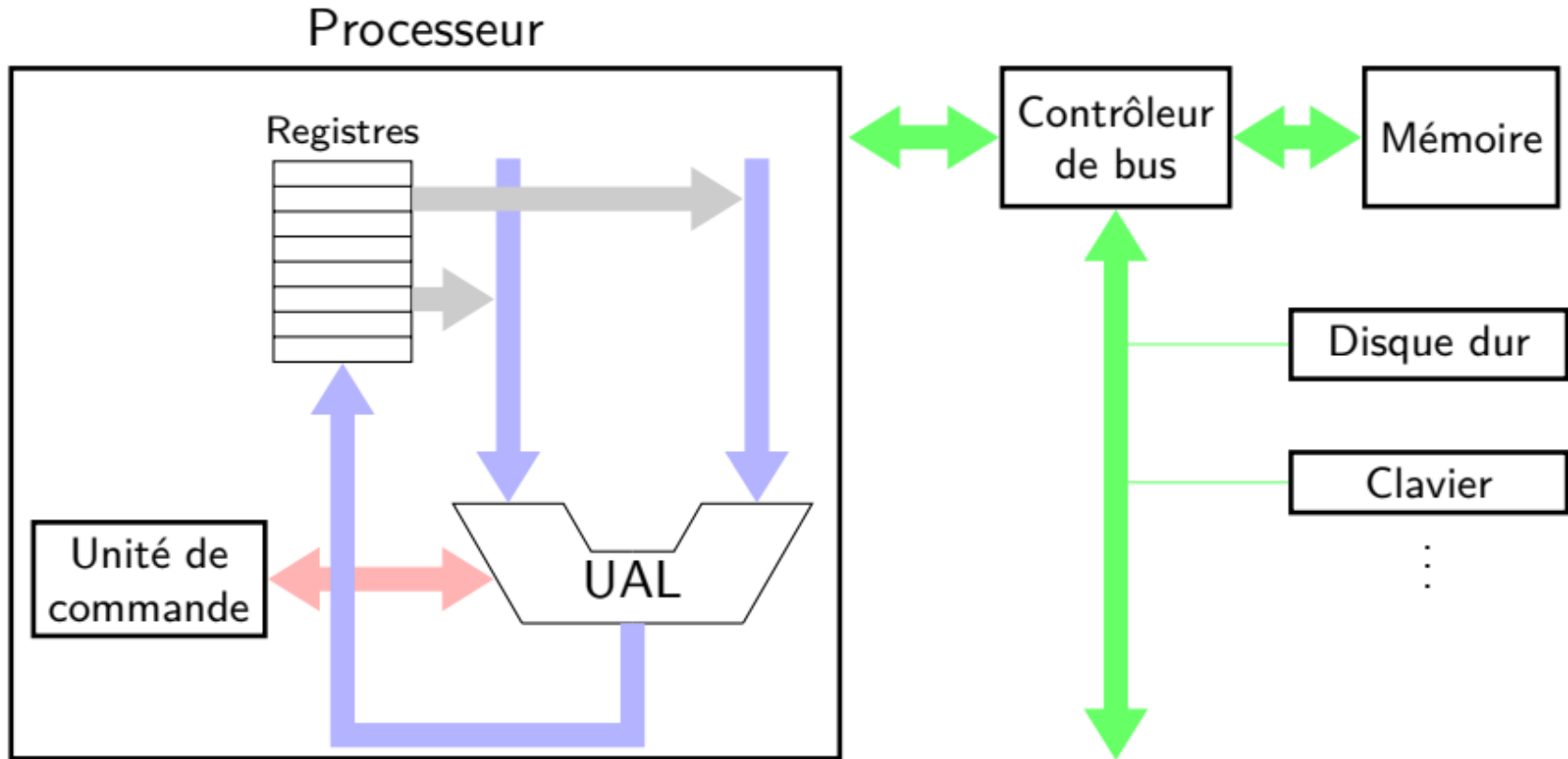


Logique séquentielle :

- Registre N bits :
 - Mémorisation d'un vecteur binaire de N bits
 - N bascules D flip-flops



Microprocesseur :

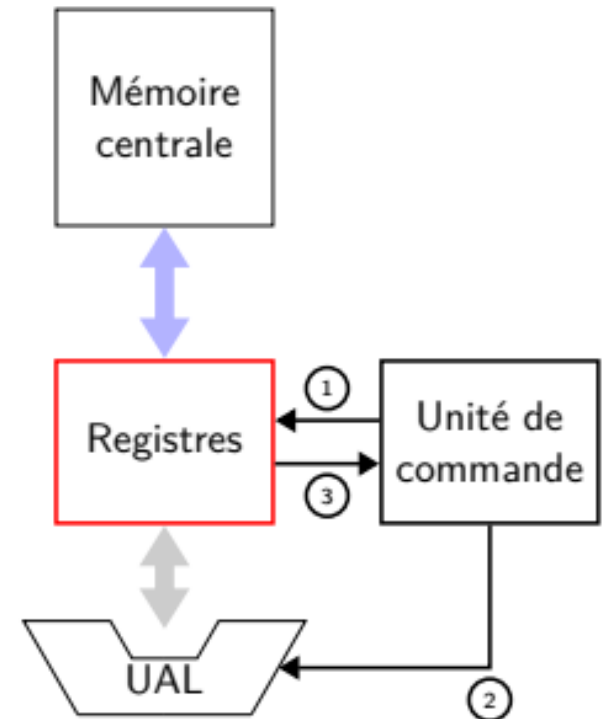


Microprocesseur :

- Les parties essentielles d'un processeur sont :
 - L'Unité Arithmétique et Logique (UAL, en anglais Arithmetic and Logical Unit - ALU), qui prend en charge les calculs arithmétiques élémentaires et les tests
 - L'unité de contrôle ou séquenceur, qui permet de synchroniser les différents éléments du processeur. En particulier, il initialise les registres lors du démarrage de la machine et il gère les interruptions
 - Les registres, qui sont des mémoires de petite taille (quelques octets), suffisamment rapides pour que l'UAL puisse manipuler leur contenu à chaque cycle de l'horloge. Un certain nombre de registres sont communs à la plupart des processeurs (compteur de programme, registre d'adresse, registre d'instruction ...)

Microprocesseur :

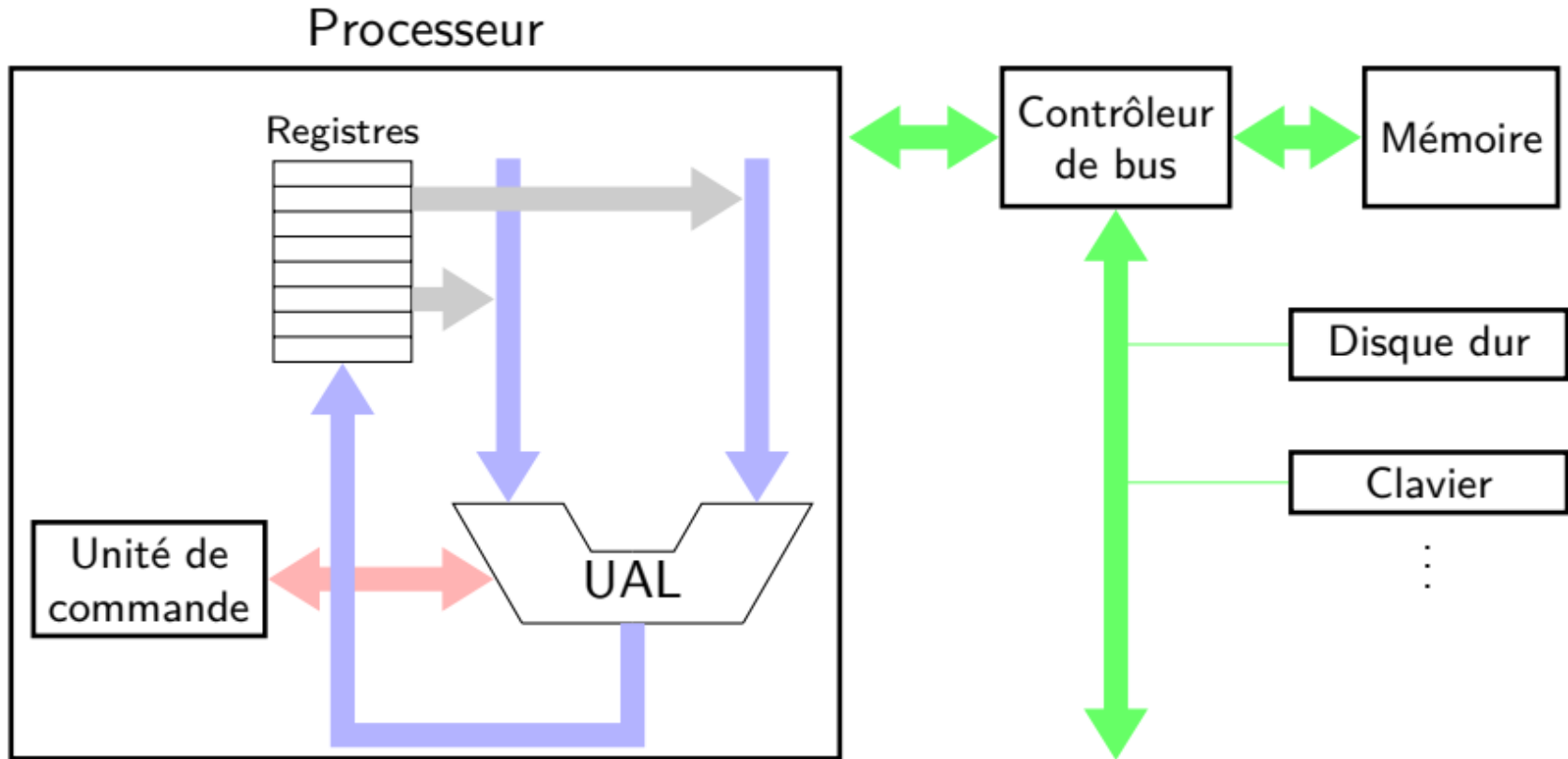
- L'unité de commande (UC) active certains registres pour :
 - Lire en mémoire
 - Ecrire en mémoire
 - Transférer des données vers l'UAL
 - Transférer des données depuis l'UAL
- L'UC commande l'action de l'UAL
- L'état des registres permet de choisir la prochaine commande



Microprocesseur :

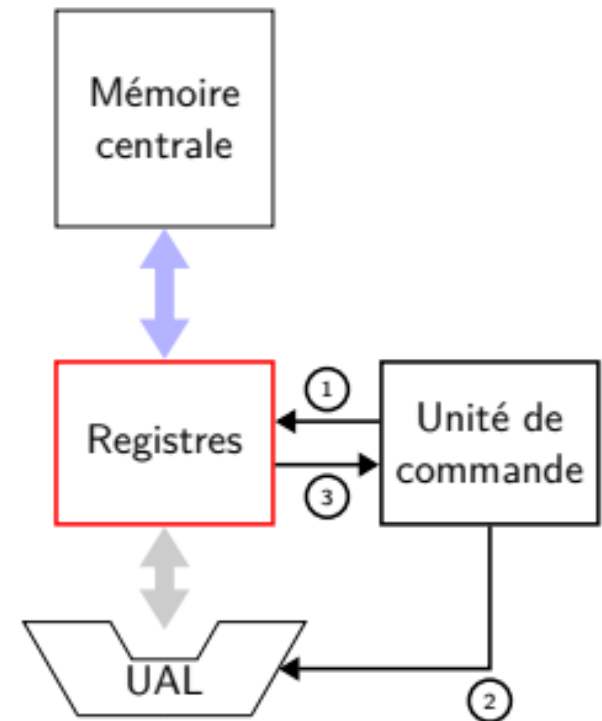
- Répéter indéfiniment (tant que le processeur est alimenté) :
 - Récupérer l'instruction courante en mémoire
 - Modifier la valeur du compteur ordinal pour qu'il désigne la prochaine instruction
 - Si besoin, localiser un mot en mémoire (donnée/prog.) (charger le registre d'adresse)
 - Si besoin, charger le mot mémoire dans un registre de données
 - Exécuter l'instruction

Mémoire :



Mémoire centrale :

- La mémoire est divisée en emplacement (des cases mémoires)
- Adresse mémoire : dans une mémoire de taille N , on a N emplacement numérotés (ou adressés) de 0 à $N-1$
- Chaque case mémoire est repéré par un numéro unique appelé adresse (souvent écrite en hexadécimal)



Mémoire :

- Opérations sur la mémoire :
 - Seul le processeur peut modifier l'état de la mémoire
 - Chaque emplacement mémoire conserve les informations que le processeur y écrit jusqu'à coupure de l'alimentation, où tout le contenu est perdu (contrairement au contenu des mémoires externes : disque dur..)

Mémoire :

- Opérations sur la mémoire :
 - Seul le processeur peut modifier l'état de la mémoire
 - On parle de mémoire vive, les seules opérations possibles :
 - écriture d'un emplacement : le processeur donne une valeur et une adresse, et la mémoire range la valeur à l'emplacement indiqué
 - Lecture d'un emplacement : le processeur demande à la mémoire la valeur contenue à l'emplacement dont il indique l'adresse, le contenu de l'emplacement auquel le processeur accède en lecture demeure inchangé

Mémoire (caractéristiques) :

- Capacité : nombre total de bits que contient la mémoire. Elle s'exprime aussi souvent en octet
- Format des données : nombre de bits que l'on peut mémoriser par case mémoire. On parle de la largeur du mot mémorisable
- Temps d'accès : temps qui s'écoule entre l'instant où a été lancée une opération de lecture/écriture en mémoire et l'instant où la première information est disponible sur le bus de données

Mémoire (caractéristiques) :

- Temps de cycle : il représente l'intervalle minimum qui doit séparer deux demandes successives de lecture ou d'écriture
- Débit : nombre maximum d'informations lues ou écrites par seconde
- Volatilité : elle caractérise la permanence des informations dans la mémoire. L'information stockée est volatile si elle risque d'être altérée par un défaut d'alimentation électrique et non volatile dans le cas contraire

Mémoire (Types) :

- Deux types distincts de mémoire :
 - Mémoires vives (RAM)
 - Mémoires mortes (ROM)
- Mémoire vive
 - Sert au stockage temporaire de données
 - En général volatiles : elles perdent leurs informations en cas de coupure d'alimentation
 - Deux grandes familles :
 - Les RAM statiques
 - Les RAM dynamiques

Mémoire (Types) :

- Mémoire vive
 - Sert au stockage temporaire de données
 - En général volatiles : elles perdent leurs informations en cas de coupure d'alimentation
 - Deux grandes familles :
 - Les RAM statiques (SRAM) : le bit mémoire est composé d'une bascule
 - Les RAM dynamiques (DRAM) : l'information est mémorisée sous la forme d'une charge électrique stockée dans un condensateur